

System description and team details

Team

Anke de Frey - u24611400

Rynhardt Gouws - u25189922

Terrence Hluyo - u25446968

Domain

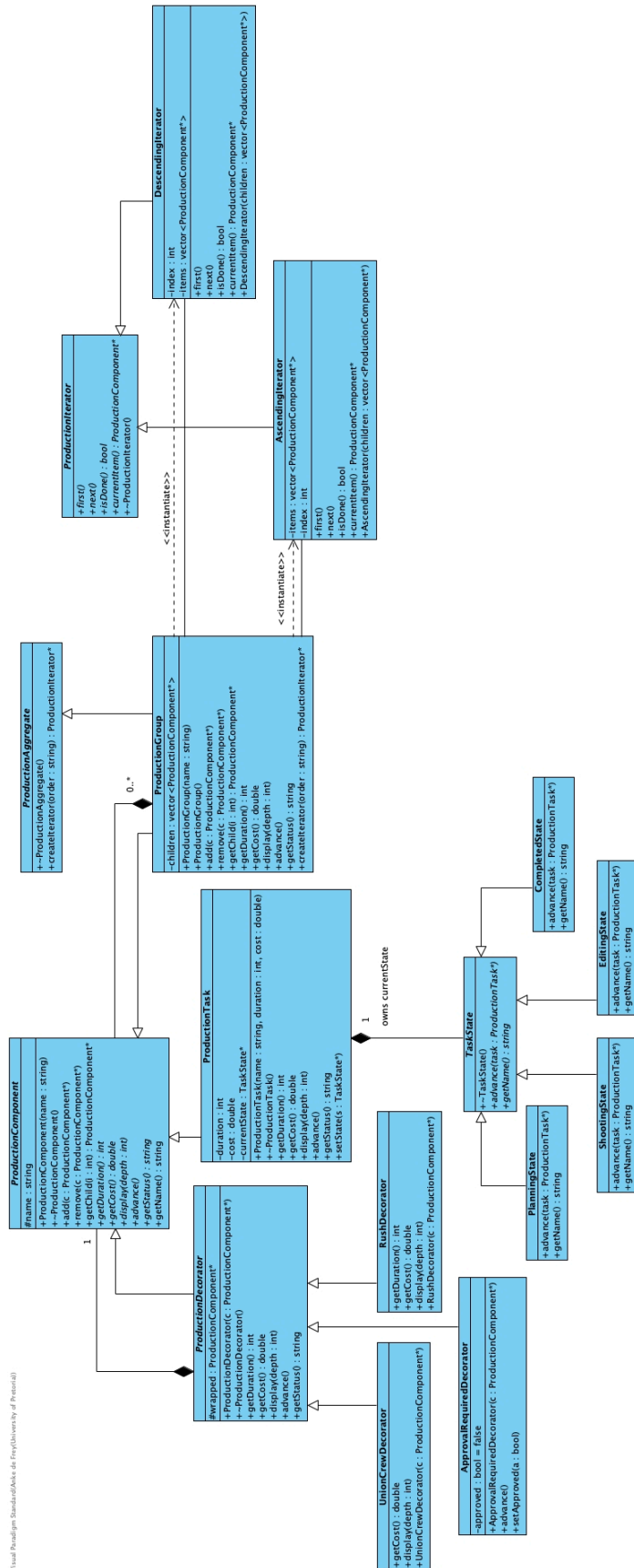
TaskForge models a film production's work hierarchy. A production ("12 Angry Dining Philosophers" in the running example) is organised into nested groups - pre production, production, post production, which may themselves contain further nested groups and individual production tasks.

Each task has a lifecycle (Planning -> Shooting -> Editing -> Completed) and may acquire additional responsibilities at runtime (a rushed schedule, certified union crew, a required approval gate) without changing its underlying type.

Problem solved

A production manager needs to treat a single task and an entire group uniformly (Composite), inspect the same hierarchy in more than one order without exposing its internal storage (Iterator), model tasks whose behaviour genuinely changes as they move through production stages (State), and add optional cost/schedule/approval responsibilities to a task at runtime, stackably, without combinatorial explosion of subclasses (Decorator)

UML Class diagram and GoF participant mapping



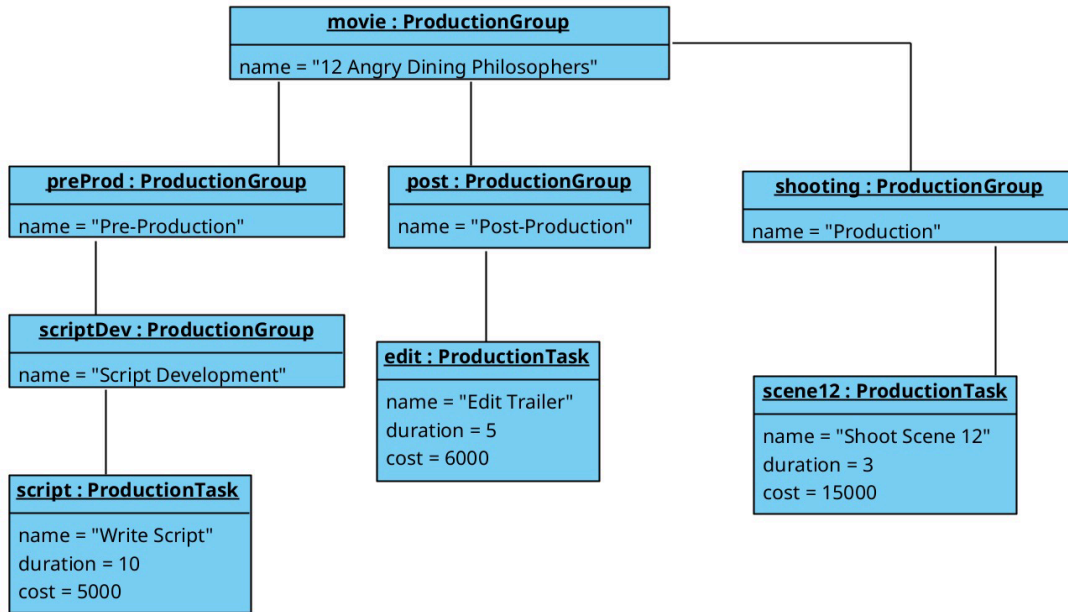
Composite	Component	ProductionComponent
Composite	Leaf	ProductionTask
Composite	Composite	ProductionGroup
Composite	Client	main.cpp
Iterator	Aggregate	ProductionAggregate
Iterator	ConcreteAggregate	ProductionGroup
Iterator	Iterator	ProductionIterator
Iterator	ConcreteIterator	AscendingIterator, Descending Iterator
Decorator	Component	ProductionComponent
Decorator	ConcreteComponent	ProductionTask
Decorator	Decorator	ProductionDecorator
Decorator	ConcreteDecorator	RushDecorator, UnionCrewDecorator, ApprovalRequiredDecorator
State	Context	ProductionTask
State	State	TaskState
State	ConcreteState	PlanningState, ShootingState, EditingState, CompletedState

Design and ownership rationale

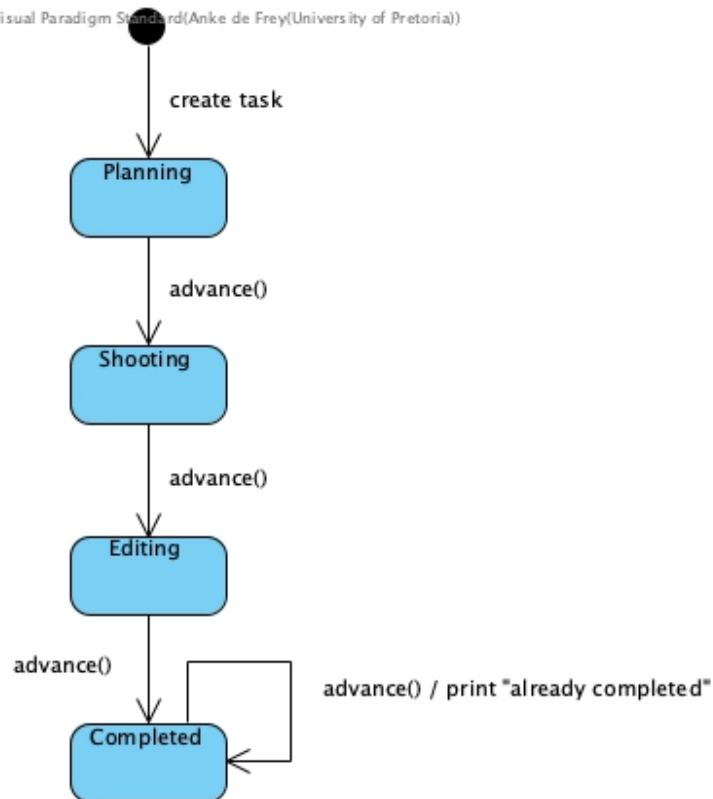
- **Composite design:** `add()`, `remove()`, and `getChild()` are included in `ProductionComponent`. This allows all components to be treated in the same way without using type checks. Leaf components simply use the default no-op versions.
- **Group calculations:** `ProductionGroup` calculates its duration and cost by adding the values of all its children. This assumes that the tasks happen one after another, rather than at the same time.
- **Decorator names:** A decorator uses the same name as the component it wraps because it represents the same task with extra responsibilities, not a separate task.
- **Iterator interface:** `ProductionAggregate` is separate from `ProductionComponent` and is only implemented by `ProductionGroup`. Only groups contain collections that need to be traversed, so leaves do not need a meaningless `createIterator()` function.
- **Iterator snapshots:** An iterator stores a copy of the group's children when it is created. Therefore, changes made to the group afterward do not affect an iterator that is already being used.
- **Group status:** `ProductionGroup::getStatus()` counts completed tasks recursively by reading the status strings returned by its children. This avoided changing the shared interface late in development, although it makes the method dependent on the current status-string format.
- **Decorator display:** Each decorator displays its own updated duration and cost. This ensures that the values include the decorator's changes instead of showing only the original task's values.
- **Completed state:** Calling `advance()` on a completed task does not change its state. Instead, it displays a message explaining that the task is already complete, which handles the invalid transition clearly.
- **Inheritance:** `ProductionTask` and `ProductionGroup` inherit from `ProductionComponent`. The decorators inherit from `ProductionDecorator`, the states inherit from `TaskState`, and the iterators inherit from `ProductionIterator`. `ProductionGroup` also implements `ProductionAggregate`.
- **Ownership:** Composition is used where one class owns another. `ProductionGroup` owns its children, `ProductionDecorator` owns its wrapped component, and `ProductionTask` owns its current state.

- **Multiplicities:** A group can own 0..* children, while a decorator has exactly 1 wrapped component and a task has exactly 1 current state.
- **Iterator associations:** AscendingIterator and DescendingIterator each have a plain association with ProductionComponent, with multiplicity 0..*. This is not composition because the iterators only store references to the components and do not own or delete them.

Object and State Diagram



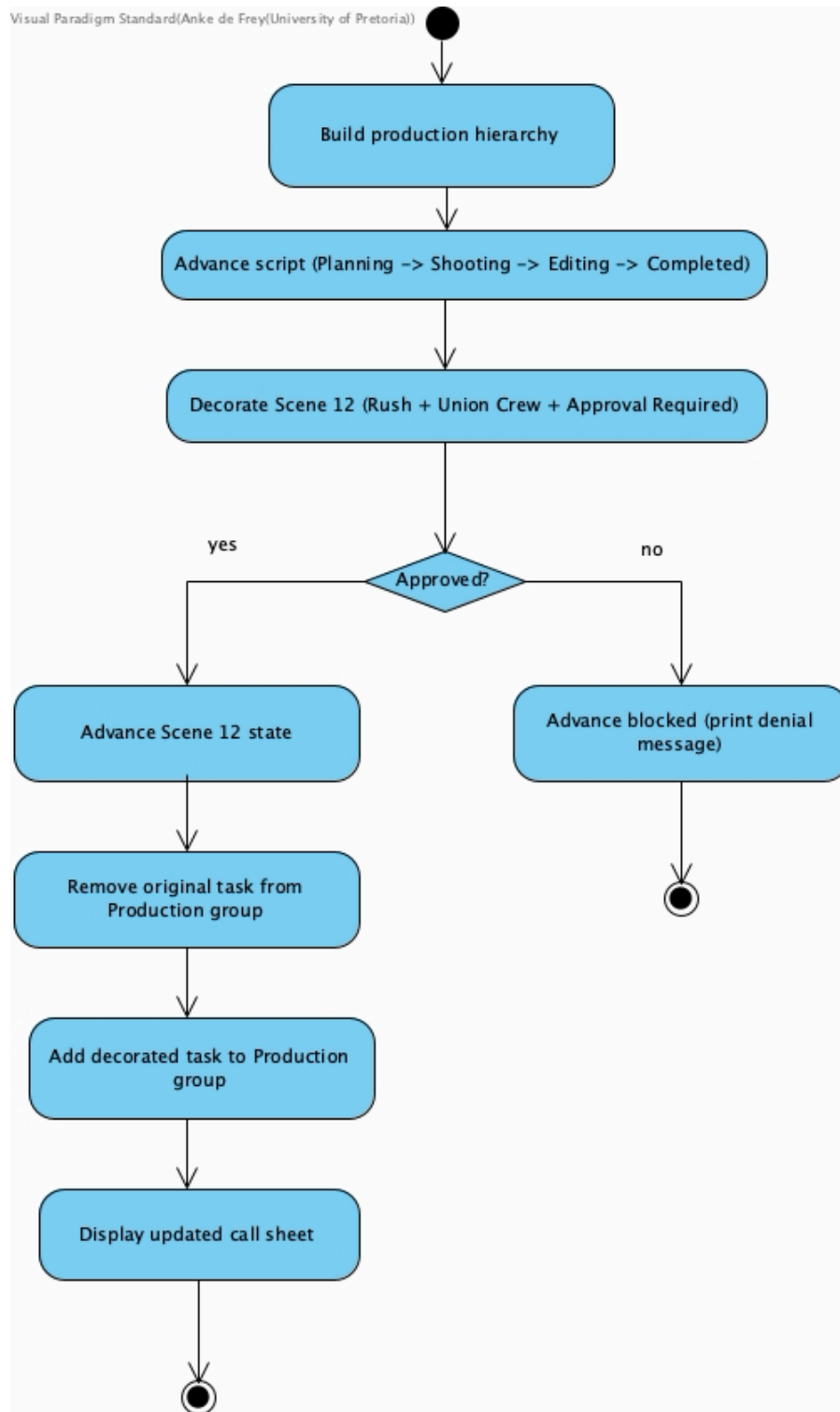
Visual Paradigm Standard(Anke de Frey(University of Pretoria))



Activity diagrams

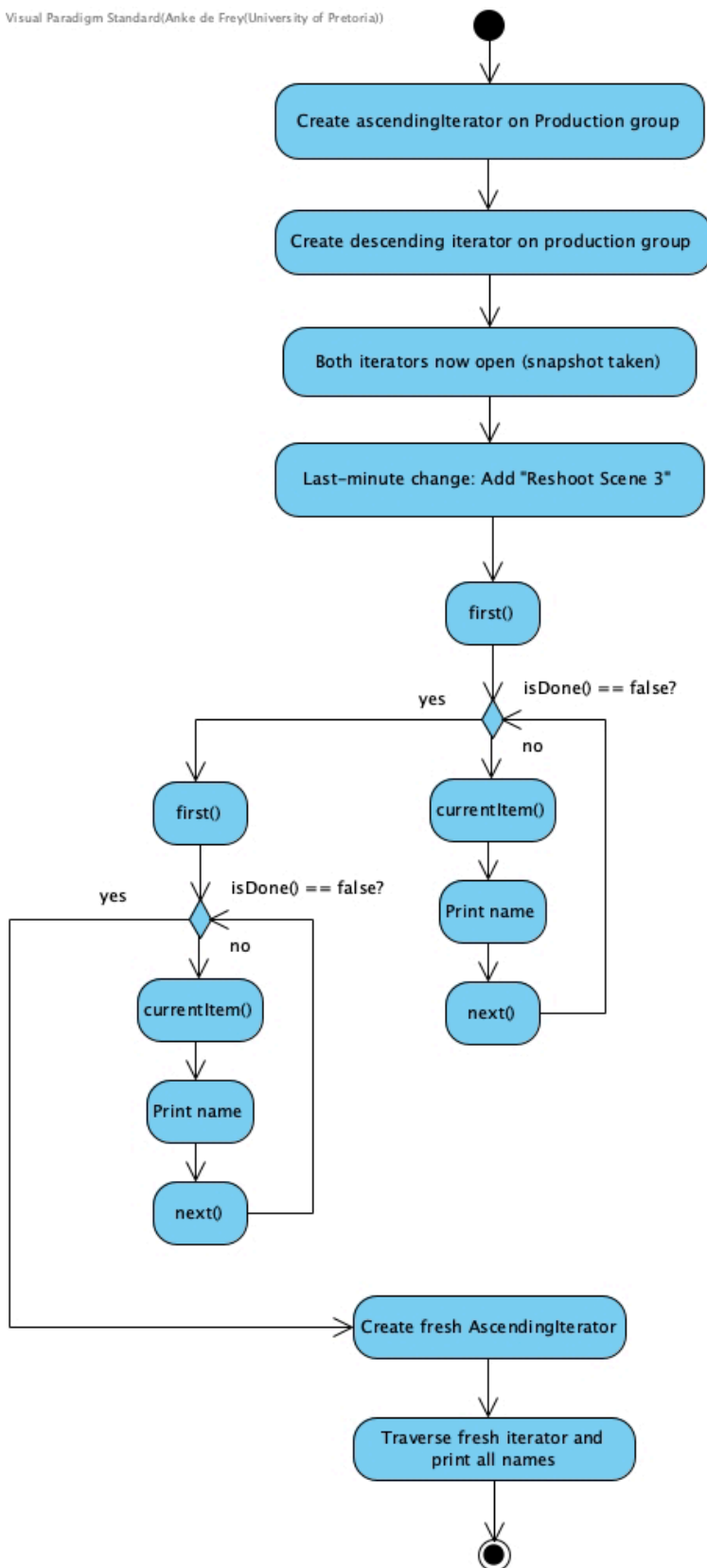
4.1)

Visual Paradigm Standard (Anke de Frey (University of Pretoria))

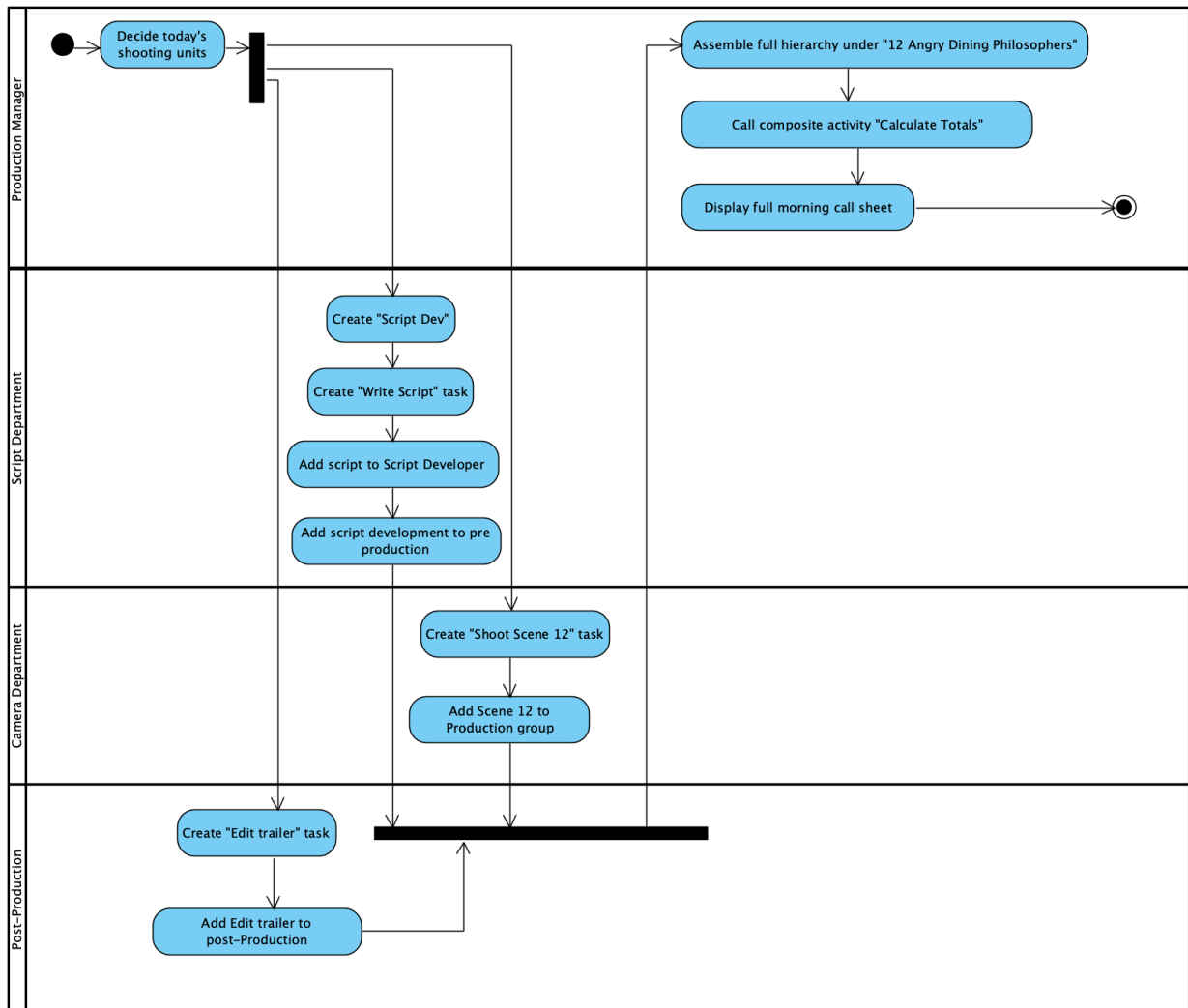


4.2)

Visual Paradigm Standard(Anke de Frey(University of Pretoria))



4.3)



Traversal modification policy and other design decisions

taskForge uses a snapshot traversal policy:

- Each iterator (AscendingIterator, DescendingIterator) copies its parent group's children vector at compile time instead of holding a live reference to it
- An iterator already in progress is therefore unaffected by any structural change made to the group after that point like additions, removals or reordering.

This is demonstrated live in the main.cpp.

- 2 independent iterators (ascending and descending are created and started over the Production group
- A new task "Reshoot Scene 3", is then added to that same group while both iterators are already open
- Both iterators subsequently run to completion without including the new task.
- A fresh iterator, created afterward, does include it - confirming the policy holds specifically for iterators that predate the change, not for the group's state in general.

Task 5

Evidence of using GDB, breakpoints/stepping and inspection

```
root@61a32f97e4d4:/app# gdb ./taskforge
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
```

```
(gdb) break ProductionTask::display
Breakpoint 1 at 0x7508: file ProductionTask.cpp, line 24.
```

A breakpoint was placed in `ProductionTask::display()` to pause whenever a task was displayed.

```
(gdb) run
Starting program: /app/taskforge
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".
==TaskForge: A day on set==

==Advancing the script through its lifecycle==
Write Script is already completed.

==A rush order comes in on Scene 12==
Blocked: approval required before this task can advance.

==Two independant traversals over Production==
Ascending (before the add):
  Shoot Scene 12
Descending (before the add):
  Shoot Scene 12
Fresh iterator, created after the add:
  Shoot Scene 12
  Reshoot Scene 3
==Final structure==
Name: 12 Angry Dining Philosophers Duration: 19 Cost: 34750 State: 1/4 tasks completed
Name: Pre-Production Duration: 10 Cost: 5000 State: 1/1 tasks completed
Name: Script Development Duration: 10 Cost: 5000 State: 1/1 tasks completed

Breakpoint 1, ProductionTask::display (this=0xaaaa3a14520, depth=3) at ProductionTask.cpp:24
24     void ProductionTask::display(int depth) {
```

Run

```
(gdb) print this->name
$1 = "Write Script"
```

The first breakpoint was for "Write Script", which was not the task being investigated, so execution continued.

```
(gdb) continue
Continuing.
Name:Write ScriptDuration: 10Cost: 5000State: Completed
Name: Production Duration: 4 Cost: 23750 State: 0/2 tasks completed
  [Union Crew]
  [Rush]

Breakpoint 1, ProductionTask::display (this=0xaaaa3a14570, depth=4) at ProductionTask.cpp:24
24     void ProductionTask::display(int depth) {
```

```
(gdb) print this->name
$2 = "Shoot Scene 12"
(gdb) print this->duration
$3 = 3
(gdb) print this->cost
$4 = 15000
```

Second hit is scene12's call, confirmed as "Shoot Scene 12"

Inspected the raw stored fields directly at the breakpoint: duration = 3 and cost = 15000

```
(gdb) bt
#0  ProductionTask::display (this=0xaaaaad3a14570, depth=4) at ProductionTask.cpp:24
#1  0x0000aaaacdb279e0 in RushDecorator::display (this=0xaaaaad3a146f0, depth=3) at RushDecorator.cpp:25
#2  0x0000aaaacdb27da8 in UnionCrewDecorator::display (this=0xaaaaad3a14730, depth=2) at UnionCrewDecorator.cpp:18
#3  0x0000aaaacdb24cec in ProductionDecorator::display (this=0xaaaaad3a14770, depth=2) at ProductionDecorator.cpp:19
#4  0x0000aaaacdb253d4 in ProductionGroup::display (this=0xaaaaad3a14480, depth=1) at ProductionGroup.cpp:58
#5  0x0000aaaacdb253d4 in ProductionGroup::display (this=0xaaaaad3a142f0, depth=0) at ProductionGroup.cpp:58
#6  0x0000aaaacdb28934 in main () at main.cpp:98
```

Backtrace (bt) shows the decorator chain:

RushDecorator::display ->

UnionCrewDecorator::display ->

ProductionDecorator::display

(the inherited fallback, since ApprovalRequiredDecorator doesn't override display()) ->

ProductionGroup::display (twice, nested) ->

main.

```
(gdb) frame 1
#1  0x0000aaaacdb279e0 in RushDecorator::display (this=0xaaaaad3a146f0, depth=3) at RushDecorator.cpp:25
25      wrapped->display(depth + 1);
(gdb) print this
$5 = (RushDecorator * const) 0xaaaaad3a146f0
(gdb) print wrapped
$6 = (ProductionComponent *) 0xaaaaad3a14570
```

The wrapped pointer in RushDecorator had the same address as the task being displayed. This confirmed that the decorator chain eventually reached the original task.

Symptom:

Shoot Scene 12 was marked as [Union Crew] and [Rush], but it still displayed its original duration of 3 and cost of 15000. The parent group's totals included the decorator changes, so the two outputs did not agree.

Cause:

Each decorator displayed its label and then called wrapped->display().

Eventually, ProductionTask::display() was called.

This function printed the task's original duration and cost fields.

Decorators do not change these original fields, so the undecorated values were displayed.

The group totals were correct because the group called getDuration() and getCost() on the outermost decorator. Virtual dispatch then applied every decorator's calculation.

Correction:

The first attempted fix changed `ProductionTask::display()` to use `getDuration()` and `getCost()`. However, this still returned the original values because this referred to the raw `ProductionTask`, not the outer decorators.

The working fix was to make each decorator display the values returned by its own getters before calling the wrapped object's `display()` function. This ensures that every decorator layer displays the duration and cost that apply at that level.

(1)

Before:

```
void ProductionTask::display(int depth) {
    cout << "Name:" << getName() << "Duration: " << this->duration << "Cost: " << this->cost << "State: " << getStatus() << endl;
}
```

After:

```
void ProductionTask::display(int depth) {
    for (int i = 0; i < depth; i++) {
        cout << " ";
    }
    cout << "Name: " << getName() << " Duration: " << getDuration() << " Cost: " << getCost() << " State: " << getStatus() << endl;
}
```

(2)

```
void ProductionDecorator::display(int depth) {
    wrapped->display(depth);
}
```

```
void ProductionDecorator::display(int depth) {
    for (int i = 0; i < depth; ++i) {
        std::cout << " ";
    }
    std::cout << "Name: " << getName() << " Duration: " << getDuration()
    << " Cost: " << getCost() << " State: " << getStatus() << std::endl;
    wrapped->display(depth + 1);
}
```

```
void RushDecorator::display(int depth) {
    for (int i = 0; i < depth; ++i) {
        std::cout << " ";
    }
    std::cout << "[Rush]" << std::endl;
    wrapped->display(depth + 1);
}
```

```

void RushDecorator::display(int depth) {
    for (int i = 0; i < depth; ++i) {
        std::cout << " ";
    }
    std::cout << "[Rush] Name: " << getName() << " Duration: " << getDuration()
    << " Cost: " << getCost() << " State: " << getStatus() << std::endl;
    wrapped->display(depth + 1);
}

```

```

void UnionCrewDecorator::display(int depth) {
    for (int i = 0; i < depth; ++i) {
        std::cout << " ";
    }
    std::cout << "[Union Crew]" << std::endl;
    wrapped->display(depth + 1);
}

```

```

void UnionCrewDecorator::display(int depth) {
    for (int i = 0; i < depth; ++i) {
        std::cout << " ";
    }
    std::cout << "[Union Crew] Name: " << getName() << " Duration: " << getDuration()
    << " Cost: " << getCost() << " State: " << getStatus() << std::endl;
    wrapped->display(depth + 1);
}

```

Output after changed:

```
==TaskForge: A day on set==

==Advancing the script through its lifecycle==
Write Script is already completed.

==A rush order comes in on Scene 12==
Blocked: approval required before this task can advance.

==Two independant traversals over Production==
Ascending (before the add):
  Shoot Scene 12
Descending (before the add):
  Shoot Scene 12
Fresh iterator, created after the add:
  Shoot Scene 12
  Reshoot Scene 3
==Final structure==
Name: 12 Angry Dining Philosophers Duration: 19 Cost: 34750 State: 1/4 tasks completed
Name: Pre-Production Duration: 10 Cost: 5000 State: 1/1 tasks completed
Name: Script Development Duration: 10 Cost: 5000 State: 1/1 tasks completed
  Name: Write Script Duration: 10 Cost: 5000 State: Completed
Name: Production Duration: 4 Cost: 23750 State: 0/2 tasks completed
  Name: Shoot Scene 12 Duration: 2 Cost: 15750 State: Shooting
    [Union Crew] Name: Shoot Scene 12 Duration: 2 Cost: 15750 State: Shooting
    [Rush] Name: Shoot Scene 12 Duration: 2 Cost: 15250 State: Shooting
      Name: Shoot Scene 12 Duration: 3 Cost: 15000 State: Shooting
  Name: Reshoot Scene 3 Duration: 2 Cost: 8000 State: Planning
Name: Post-Production Duration: 5 Cost: 6000 State: 0/1 tasks completed
  Name: Edit Trailer Duration: 5 Cost: 6000 State: Planning
```

Valgrind evidence

```
root@5f2f77c5e20b:/app# valgrind --leak-check=full --show-leak-kinds=all ./taskforge
==51== Memcheck, a memory error detector
==51== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==51== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==51== Command: ./taskforge
==51==
==TaskForge: A day on set==

==Advancing the script through its lifecycle==
Write Script is already completed.

==A rush order comes in on Scene 12==
Blocked: approval required before this task can advance.

==Two independant traversals over Production==
Ascending (before the add):
  Shoot Scene 12
Descending (before the add):
  Shoot Scene 12
Fresh iterator, created after the add:
  Shoot Scene 12
  Reshoot Scene 3
==Final structure==
Name: 12 Angry Dining Philosophers Duration: 19 Cost: 34750 State: 1/4 tasks completed
Name: Pre-Production Duration: 10 Cost: 5000 State: 1/1 tasks completed
Name: Script Development Duration: 10 Cost: 5000 State: 1/1 tasks completed
  Name: Write Script Duration: 10 Cost: 5000 State: Completed
Name: Production Duration: 4 Cost: 23750 State: 0/2 tasks completed
  Name: Shoot Scene 12 Duration: 2 Cost: 15750 State: Shooting
    [Union Crew] Name: Shoot Scene 12 Duration: 2 Cost: 15750 State: Shooting
      [Rush] Name: Shoot Scene 12 Duration: 2 Cost: 15250 State: Shooting
        Name: Shoot Scene 12 Duration: 3 Cost: 15000 State: Shooting
      Name: Reshoot Scene 3 Duration: 2 Cost: 8000 State: Planning
  Name: Post-Production Duration: 5 Cost: 6000 State: 0/1 tasks completed
    Name: Edit Trailer Duration: 5 Cost: 6000 State: Planning
==51==
==51== HEAP SUMMARY:
==51==   in use at exit: 0 bytes in 0 blocks
==51==   total heap usage: 57 allocs, 57 frees, 75,364 bytes allocated
==51==
==51== All heap blocks were freed -- no leaks are possible
==51==
==51== For lists of detected and suppressed errors, rerun with: -s
==51== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Github repo and workflow reflection

Repo link:

Workflow reflection

Work was split by pattern, each on its own branch:

- Rynhardt owned Composite and Iterator (ProductionGroup, ProductionAggregate, AscendingIterator, DescendingIterator),
- Terrence owned Decorator (ProductionDecorator, RushDecorator, UnionCrewDecorator, ApprovalRequiredDecorator)
- Anke owned State (TaskState and its four concrete states) plus main.cpp and final integration.
- Integration surfaced genuine issues that were resolved through the git history rather than around it - for example, a merge conflict in ProductionTask.cpp where two branches had independently written different halves of the same constructor, resolved by combining both halves rather than picking one side

Merge pull request #5 from Ankedefrey/feature/state

Ankedefrey authored yesterday

Verified e623062

Implement ProductionDecorator methods and constructor

u25446968-sudo authored yesterday

Verified 690a643

Added comments to an error

rynhardt406-spec committed yesterday

d67b90a

Merge branch 'Composite-Real'

rynhardt406-spec committed yesterday

48ba184

Team contribution

Anke

- UML class diagram
- Implemented the state hierarchy (TaskState and its four concrete states)
- Wrote main.cpp
- Led debugging and integration across all branches, including gdb investigation and correction, valgrind verification, docker setup, makefile
- State diagram
- 1 Activity diagram
- PDF assembly

Terrence

- Implemented the decorator hierarchy (ProductionDecorator, RushDecorator, UnionCrewDecorator, ApprovalRequiredDecorator)
- 2 Activity diagrams

Rynhardt

- Initial scenario
- Implemented the Composite hierarchy (ProductionComponent, ProductionGroup, ProductionTask core operations) and Iterator implementation (ProductionAggregate, ProductionIterator, AscendingIterator, DescendingIterator)
- Object diagram